西安交通大学实验报告

课程 Rust 程序设计 实验名称__实验三____ 第 1 页 共 37 页

系 别_____计算机科学与技术_____ 实 验 日 期 2026 年 5 月 20 日
专业班级_____组别_____6_____ 实 验 报 告 日 期 2026 年 5 月 20 日
姓 名_____学号 报 告 退 发（订正 、 重做 ）
同 组 人____、、（该实验独立完成）_____
教 师 审 批 签 字

实验名称：实验三

一. 题目

任务一：用户注册系统（SQLite）

- 创建一个 SQLite 数据库 users.db
- 建立 users 表：id 主键、username 唯一、password、created_at 时间戳
- 编写 Rust 程序实现用户注册功能，插入测试数据（至少 5 条）
- 实现按用户名查询用户信息的功能

任务二：日志记录系统（MySQL）

- 连接本地 MySQL 数据库创建 app_logs 表
- 字段包含：log_id 自增、log_level ENUM、content、created_at
- 编写日志写入函数，支持插入不同级别的日志
- 实现查询最近 24 小时的 ERROR 级别日志

任务三：商品库存管理（PostgreSQL）

- 在 PostgreSQL 中创建 products 表

- 字段：商品 ID、名称、库存数量、最后更新时间
- 实现库存扣减功能：购买商品后更新库存
- 编写查询库存量低于 10 件的商品功能

任务四：数据统计报表（MySQL）

- 基于订单表生成每日销售报表
- 统计：订单总数、销售总额、平均客单价
- 使用数据库视图存储统计逻辑
- 实现按日期范围查询统计结果

二. 环境

• 操作系统：Windows 11

• 运行平台：原生 Windows 桌面

• 内核版本：Windows NT 10.0.26200

• 处理器：13th Gen Intel(R) Core(TM) i5-13500HX

• CPU 核心：14 物理核心 / 20 逻辑 CPU

• 内存：15.7 GiB

• 交换空间：约 20 GiB

• 容器运行时：Docker 28.4.0 (MySQL 8.0 / PostgreSQL 16)

• Rust 工具链：stable-x86_64-pc-windows-msvc

• 编译器版本：rustc 1.94.0

• 构建工具版本：cargo 1.94.0

三. 实验步骤

任务一：用户注册系统

```
use std::io::{self, BufRead, Write};



use anyhow::Result;

use chrono::{DateTime, Local};

use rusqlite::{Connection, OptionalExtension};



#[derive(Debug)]

#[allow(dead_code)]

struct User {

    id: i64,

    username: String,

    password: String,

    created_at: DateTime<Local>,

}



/// 打开数据库连接，并在不存在时创建 users 表。

fn init_db(path: &str) -> Result<Connection> {

    let conn = Connection::open(path)?;

    conn.execute(

        "CREATE TABLE IF NOT EXISTS users (

            id INTEGER PRIMARY KEY AUTOINCREMENT,
```

```
            username TEXT NOT NULL UNIQUE,
            password TEXT NOT NULL,
            created_at DATETIME NOT NULL
        )",
        [],
    )?;
    Ok(conn)
}

/// 注册一个新用户。username 已存在时应返回错误（依赖 UNIQUE 约束）。
fn register_user(conn: &Connection, username: &str, password: &str) -> Result<i64> {
    let now = Local::now();
    conn.execute(
        "INSERT INTO users (username, password, created_at) VALUES (?1, ?2, ?3)",
        rusqlite::params![username, password, now],
    )?;
    Ok(conn.last_insert_rowid())
}

/// 根据用户名查询用户信息。
fn find_user_by_username(conn: &Connection, username: &str) -> Result<Option<User>> {
    conn.query_row(
```

```
        "SELECT id, username, password, created_at FROM users WHERE username = ?1",
        rusqlite::params![username],
        |row| {
            Ok(User {
                id : row.get(0)?,
                username : row.get(1)?,
                password : row.get(2)?,
                created_at : row.get(3)?,
            })
        },
    )
    .optional()
    .map_err(Into::into)
}


/// 从终端读一行，去掉首尾空白。EOF（Ctrl+Z / Ctrl+D）时返回 None。
fn read_line(prompt: &str) -> Result<Option<String>> {
    print!("{prompt}");
    io::stdout().flush()?;
    let mut buf = String::new();
    let n = io::stdin().lock().read_line(&mut buf)?;
    if n == 0 {
```

```
        return Ok(None);
    }
    Ok(Some(buf.trim().to_string()))
}

/// 首次运行时填充 5 条种子数据；已存在则静默跳过。
fn seed_users(conn: &Connection) {
    let seed = [
        ("alice", "pass_alice"),
        ("bob", "pass_bob"),
        ("carol", "pass_carol"),
        ("dave", "pass_dave"),
        ("eve", "pass_eve"),
    ];
    for (u, p) in seed {
        let _ = register_user(conn, u, p);
    }
}

fn action_register(conn: &Connection) -> Result<()> {
    let username = match read_line("用户名: ")? {
        Some(s) if !s.is_empty() => s,
```

```
        _ => {
            println!("用户名不能为空。");
            return Ok(());
        }
    };
    let password = match read_line("密码:   ")? {
        Some(s) if !s.is_empty() => s,
        _ => {
            println!("密码不能为空。");
            return Ok(());
        }
    };
    match register_user(conn, &username, &password) {
        Ok(id) => println!("✓ 注册成功: id={id}, username={username}"),
        Err(e) => println!("✗ 注册失败: {e}"),
    }
    Ok(())
}

fn action_query(conn: &Connection) -> Result<()> {
    let username = match read_line("要查询的用户名: ")? {
        Some(s) if !s.is_empty() => s,
```

```
        _ => {
            println!("用户名不能为空。");
            return Ok(());
        }
    };
    match find_user_by_username(conn, &username)? {
        Some(u) => {
            println!("✓ 找到用户:");
            println!("  id         = {}", u.id);
            println!("  username   = {}", u.username);
            println!("  password   = {}", u.password);
            println!("  created_at = {}", u.created_at.format("%Y-%m-%d %H:%M:%S"));
        }
        None => println!("✗ 未找到用户: {username}"),
    }
    Ok(())
}

fn main() -> Result<()> {
    let conn = init_db("users.db")?;
    seed_users(&conn);
```

```
println!("=== 用户注册系统 (SQLite) ===");

loop {

    println!();

    println!("[1] 注册新用户");

    println!("[2] 按用户名查询");

    println!("[q] 退出");

    let choice = match read_line("> ")? {

        Some(s) => s,

        None => {

            println!("\n 再见。");

            break;

        }

    };

    match choice.as_str() {

        "1" => action_register(&conn)?,

        "2" => action_query(&conn)?,

        "q" | "Q" | "quit" | "exit" => {

            println!("再见。");

            break;

        }

        "" => continue,

        other => println!("无效选项: {other}"),
```

```
    }
  }
  Ok(())
}
```

分析：以 rusqlite、 chrono、anyhow 三个 crate 为基础，开启 chrono feature 让 DateTime<Local> 可直接作为参数绑定。

init_db 用 Connection::open 打开 users.db 并执行 SQL 建表，将 username 设为 UNIQUE 让数据库层拦截重复注册。

register_user 采用参数化 SQL VALUES (?1, ?2, ?3) 配合 rusqlite::params! 宏绑定值，既防止注入也省去类型转换，插入后返回 conn.last_insert_rowid() 作为新行 id；遇到 UNIQUE 冲突的错误经 ? 抛出后由调用方匹配处理。

find_user_by_username 用 conn.query_row 配合闭包按列号 row.get(N)? 映射成 User，再链式调用 OptionalExtension::optional() 把无结果的 QueryReturnedNoRows 错误转为 Ok(None)，符合找不到时返回 None 的语义。

main 函数实现交互菜单：启动时通过 seed_users 静默灌入 5 条种子数据满足实验要求，随后进入循环，用封装的 read_line 读输入，按 1/2/q 分发到注册或查询动作。

任务二：日志记录系统

```
use anyhow::Result;
use chrono::{DateTime, Duration, Local, NaiveDateTime, TimeZone};
use mysql::prelude::*;
```

```rust
use mysql::{params, Pool};



const DB_URL: &str = "mysql://lab3:lab3pass@127.0.0.1:3306/lab3";



#[derive(Debug, Clone, Copy)]

enum LogLevel {

    Info,

    Warn,

    Error,

    Debug,

}



impl LogLevel {

    fn as_str(&self) -> &'static str {

        match self {

            LogLevel::Info => "INFO",

            LogLevel::Warn => "WARN",

            LogLevel::Error => "ERROR",

            LogLevel::Debug => "DEBUG",

        }

    }

}
```

```
#[derive(Debug)]
#[allow(dead_code)]
struct LogRow {
    log_id: u64,
    log_level: String,
    content: String,
    created_at: DateTime<Local>,
}

/// 建表
fn init_schema(pool: &Pool) -> Result<()> {
    let mut conn = pool.get_conn()?;
    conn.query_drop(
        r"CREATE TABLE IF NOT EXISTS app_logs (
            log_id BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
            log_level ENUM('INFO','WARN','ERROR','DEBUG') NOT NULL,
            content TEXT NOT NULL,
            created_at DATETIME NOT NULL
        )",
    )?;
    Ok(())
```

```rust
}


/// 以指定时间戳写入一条日志。
fn write_log_at(pool: &Pool, level: LogLevel, content: &str, when: DateTime<Local>) -> Result<()> {
    let mut conn = pool.get_conn()?;
    conn.exec_drop(
        "INSERT INTO app_logs (log_level, content, created_at) VALUES (:level, :content, :ts)",
        params! {
            "level" => level.as_str(),
            "content" => content,
            "ts" => when.format("%Y-%m-%d %H:%M:%S").to_string(),
        },
    )?;
    Ok(())
}


/// 以当前时间写入一条日志。
#[allow(dead_code)]
fn write_log(pool: &Pool, level: LogLevel, content: &str) -> Result<()> {
    write_log_at(pool, level, content, Local::now())
}
```

```
/// 查询最近 24 小时内 ERROR 级别日志。
fn query_recent_errors(pool: &Pool) -> Result<Vec<LogRow>> {
    let mut conn = pool.get_conn()?;
    let rows = conn.query_map(
        r"SELECT log_id, log_level, content,
                 DATE_FORMAT(created_at, '%Y-%m-%d %H:%i:%s')
          FROM app_logs
          WHERE log_level = 'ERROR'
            AND created_at >= NOW() - INTERVAL 24 HOUR
          ORDER BY created_at DESC",
        |(log_id, log_level, content, ts_str): (u64, String, String, String)| {
            let naive = NaiveDateTime::parse_from_str(&ts_str, "%Y-%m-%d %H:%M:%S").unwrap();
            let created_at = Local.from_local_datetime(&naive).unwrap();
            LogRow { log_id, log_level, content, created_at }
        },
    )?;
    Ok(rows)
}

fn main() -> Result<()> {
    let pool = Pool::new(DB_URL)?;
    init_schema(&pool)?;
```

```
pool.get_conn()?.query_drop("TRUNCATE TABLE app_logs")?;

let now = Local::now();
// (相对当前的偏移小时, 等级, 内容)
let samples: &[(i64, LogLevel, &str)] = &[
    // 24 小时窗口之外
    (-72, LogLevel::Info,  "三天前：服务启动"),
    (-50, LogLevel::Error, "两天前：磁盘 IO 错误"),
    (-30, LogLevel::Warn,  "30 小时前：内存占用偏高"),
    (-26, LogLevel::Error, "26 小时前：缓存命中率骤降"),
    (-25, LogLevel::Error, "25 小时前：上游 API 超时"),
    // 24 小时窗口之内
    (-23, LogLevel::Info,  "23 小时前：服务重启完成"),
    (-20, LogLevel::Warn,  "20 小时前：磁盘占用 80%"),
    (-18, LogLevel::Error, "18 小时前：数据库连接失败"),
    (-12, LogLevel::Debug, "12 小时前：调试信息 x=42"),
    (-8,  LogLevel::Error, "8 小时前：下游服务超时"),
    (-5,  LogLevel::Warn,  "5 小时前：CPU 使用率 90%"),
    (-3,  LogLevel::Error, "3 小时前：支付回调签名校验失败"),
    (-1,  LogLevel::Info,  "1 小时前：每小时定时任务完成"),
    (0,   LogLevel::Error, "刚刚：消息队列消费者掉线"),
```

```
    ];

    for (delta_h, level, msg) in samples {
        write_log_at(&pool, *level, msg, now + Duration::hours(*delta_h))?;
    }

    let errs = query_recent_errors(&pool)?;
    println!("最近 24 小时 ERROR 共 {} 条：", errs.len());
    for row in errs {
        println!("  {:?}", row);
    }
    Ok(())
}
```

分析：以 mysql、anyhow、chrono 三个 crate 为基础，DB_URL 常量指向 docker 容器中的 MySQL 服务。LogLevel 设计为 enum 并配套 as_str() 方法，把日志级别从字符串字面量收紧为编译期可枚举的类型，避免写入时拼错。init_schema 通过 query_drop 建表，将 log_level 列声明为 ENUM('INFO','WARN','ERROR','DEBUG')，让数据库层兜底校验枚举值的合法性。write_log_at 使用 params!{} 命名参数绑定，时间戳先按 %Y-%m-%d %H:%M:%S 格式化为字符串再写入 DATETIME 列，规避 mysql crate 在原生时间类型上可能出现的时区歧义。query_recent_errors 把 24 小时窗口的判定下沉到 SQL，用 created_at >= NOW() - INTERVAL 24 HOUR 让数据库以服务器时间为

准，避免客户端时钟漂移导致边界误差；返回的时间列同样先经 DATE_FORMAT 转为定长字符串，再在 query_map 闭包里用 NaiveDateTime::parse_from_str 配合 Local.from_local_datetime 还原成 DateTime<Local>，类型映射稳定。main 中先 TRUNCATE 清空旧数据，再批量写入 14 条样本，故意横跨 -72h 到 0h 这一时间段，使 24 小时窗口内外都覆盖到不同级别的日志，从而可以直接验证过滤条件是否正确。

任务三：商品库存管理

```
use std::io::{self, BufRead, Write};

use anyhow::{anyhow, Result};

use chrono::{DateTime, Local};

use postgres::{Client, NoTls};



const DB_URL: &str = "host=127.0.0.1 port=5432 user=lab3 password=lab3pass dbname=lab3";



#[derive(Debug)]

#[allow(dead_code)]

struct Product {

    id: i32,

    name: String,

    stock: i32,

    updated_at: DateTime<Local>,
```

```
}



fn connect() -> Result<Client> {

    Ok(Client::connect(DB_URL, NoTls)?)

}



/// 建表 + 插入若干样例数据（如果表为空）。

fn init_schema(client: &mut Client) -> Result<()> {

    client.batch_execute(

        "CREATE TABLE IF NOT EXISTS products (

            id         SERIAL PRIMARY KEY,

            name       TEXT NOT NULL,

            stock      INTEGER NOT NULL,

            updated_at TIMESTAMPTZ NOT NULL DEFAULT NOW()

        );

        INSERT INTO products (name, stock)

        SELECT * FROM (VALUES

            ('面包', 25),

            ('牛奶', 8),

            ('鸡蛋', 30),

            ('黄油', 5),

            ('酸奶', 3)
```

```
        ) AS seed(name, stock)
        WHERE NOT EXISTS (SELECT 1 FROM products);",
    )?;
    Ok(())
}

/// 扣减库存：qty > 0；当前库存不足时返回错误，不能扣成负数。
fn decrement_stock(client: &mut Client, product_id: i32, qty: i32) -> Result<i32> {
    if qty <= 0 {
        return Err(anyhow!("扣减数量必须为正"));
    }
    let row = client.query_opt(
        "UPDATE products
         SET stock = stock - $1, updated_at = NOW()
         WHERE id = $2 AND stock >= $1
         RETURNING stock",
        &[&qty, &product_id],
    )?;

    match row {
        Some(r) => Ok(r.get::<_, i32>(0)),
        None => Err(anyhow!("商品 {} 不存在或库存不足", product_id)),
```

```rust
    }
}


/// 查询库存低于 10 件的商品。
fn list_low_stock(client: &mut Client, threshold: i32) -> Result<Vec<Product>> {
    // TODO:
    //   SELECT id, name, stock, updated_at FROM products
    //   WHERE stock < $1
    //   ORDER BY stock ASC;
    //   使用 client.query(sql, &[&threshold])? 遍历构造 Product。
    let rows = client.query(
        "SELECT id, name, stock, updated_at FROM products
         WHERE stock < $1
         ORDER BY stock ASC",
        &[&threshold],
    )?;

    let products = rows
        .iter()
        .map(|r| Product {
            id: r.get("id"),
            name: r.get("name"),
```

```
            stock: r.get("stock"),
            updated_at: r.get("updated_at"),
        })
        .collect();
    Ok(products)
}

/// 查询所有商品。
fn list_all(client: &mut Client) -> Result<Vec<Product>> {
    let rows = client.query(
        "SELECT id, name, stock, updated_at FROM products ORDER BY id",
        &[],
    )?;
    Ok(rows
        .iter()
        .map(|r| Product {
            id: r.get("id"),
            name: r.get("name"),
            stock: r.get("stock"),
            updated_at: r.get("updated_at"),
        })
        .collect())
```

```rust
}


/// 美观打印商品列表。
fn print_products(title: &str, items: &[Product]) {
    println!("{title}（共 {} 件）：", items.len());
    for p in items {
        println!(
            "  id={:<3} 名称={:<6} 库存={:<5} 更新于 {}",
            p.id,
            p.name,
            p.stock,
            p.updated_at.format("%Y-%m-%d %H:%M:%S")
        );
    }
}


/// 从终端读一行，去掉首尾空白。EOF（Ctrl+Z / Ctrl+D）时返回 None。
fn read_line(prompt: &str) -> Result<Option<String>> {
    print!("{prompt}");
    io::stdout().flush()?;
    let mut buf = String::new();
    let n = io::stdin().lock().read_line(&mut buf)?;
```

```
    if n == 0 {
        return Ok(None);
    }
    Ok(Some(buf.trim().to_string()))
}

fn action_buy(client: &mut Client) -> Result<()> {
    let id: i32 = match read_line("商品 id: ")?.as_deref() {
        Some(s) if !s.is_empty() => match s.parse() {
            Ok(n) => n,
            Err(_) => {
                println!("商品 id 必须是整数。");
                return Ok(());
            }
        },
        _ => {
            println!("商品 id 不能为空。");
            return Ok(());
        }
    };
    let qty: i32 = match read_line("购买数量: ")?.as_deref() {
        Some(s) if !s.is_empty() => match s.parse() {
```

```
        Ok(n) => n,
        Err(_) => {
            println!("数量必须是整数。");
            return Ok(());
        }
    },
    _ => {
        println!("数量不能为空。");
        return Ok(());
    }
};

match decrement_stock(client, id, qty) {
    Ok(remain) => println!("✓ 购买成功：商品 {id} 剩余库存 = {remain}"),
    Err(e) => {
        println!("✗ 购买失败：{e}");
        return Ok(());
    }
}

let low = list_low_stock(client, 10)?;
print_products("库存低于 10 件的商品", &low);
```

```
    Ok(())
}

fn main() -> Result<()> {
    let mut client = connect()?;
    init_schema(&mut client)?;

    println!("=== 商品库存管理 (PostgreSQL) ===");
    loop {
        println!();
        println!("[1] 查看所有商品");
        println!("[2] 购买商品");
        println!("[q] 退出");
        let choice = match read_line("> ")? {
            Some(s) => s,
            None => {
                println!("\n 再见。");
                break;
            }
        };
        match choice.as_str() {
            "1" => {
```

```
            let all = list_all(&mut client)?;
            print_products("所有商品", &all);
        }
        "2" => action_buy(&mut client)?,
        "q" | "Q" | "quit" | "exit" => {
            println!("再见。");
            break;
        }
        "" => continue,
        other => println!("无效选项: {other}"),
    }
  }
  Ok(())
}
```

分析：基于 postgres crate 提供的同步阻塞接口，通过 Client::connect 配合 NoTls 建立连接。init_schema 使用 batch_execute 一次性发送建表和样例数据插入两条语句，并在 INSERT 上加 WHERE NOT EXISTS 守护，使种子数据只在表为空时灌入。decrement_stock 是本任务的核心：把判断库存是否充足和实际扣减合并到一条 UPDATE products SET stock = stock - $1 WHERE id = $2 AND stock >= $1 RETURNING stock，由数据库原子地完成检查与更新，规避了先 SELECT 再 UPDATE 在并发场景下可能出现的 TOCTOU 问题；如果条件不满足，query_opt 返回 None，再统一转换成 anyhow 错误抛出，库存永远不会被扣

成负数。list_low_stock 通过 WHERE stock < $1 ORDER BY stock ASC 按库存升序展示低库存商品，便于优先关注最紧张的品类；行映射时使用 r.get("id") 这类列名形式而非下标，可读性更好。交互层延续任务一的 read_line 加循环菜单结构，购买动作执行成功后立刻调用 list_low_stock 输出最新的低库存清单，把扣减和库存监控串成一个完整流程。

任务四：日志输出

```
use anyhow::Result;

use chrono::NaiveDate;

use mysql::prelude::*;

use mysql::{params, Pool};



const DB_URL: &str = "mysql://lab3:lab3pass@127.0.0.1:3306/lab3";



#[derive(Debug)]

#[allow(dead_code)]

struct DailyReportRow {

    day: NaiveDate,

    order_count: i64,

    total_amount: f64,

    avg_amount: f64,

}
```

```
/// 建表 orders + 视图 v_daily_sales_report。
fn init_schema(pool: &Pool) -> Result<()> {
    let mut conn = pool.get_conn()?;
    conn.query_drop(
        r"CREATE TABLE IF NOT EXISTS orders (
            order_id   BIGINT UNSIGNED AUTO_INCREMENT PRIMARY KEY,
            customer   VARCHAR(64) NOT NULL,
            amount     DECIMAL(10, 2) NOT NULL,
            created_at DATETIME NOT NULL
        )",
    )?;
    conn.query_drop(
        r"CREATE OR REPLACE VIEW v_daily_sales_report AS
          SELECT DATE(created_at) AS day,
                 COUNT(*)        AS order_count,
                 SUM(amount)     AS total_amount,
                 AVG(amount)     AS avg_amount
          FROM orders
          GROUP BY DATE(created_at)",
    )?;
    Ok(())
```

```
}


/// 插入一些样例订单数据，便于看到报表效果。
fn seed_orders(pool: &Pool) -> Result<()> {
    let mut conn = pool.get_conn()?;

    // 实验演示用：每次跑都重置订单表，保证报表输出可复现。
    conn.query_drop("TRUNCATE TABLE orders")?;

    let orders: &[(&str, &str, f64)] = &[
        ("2026-05-10 09:15:00", "alice",  199.00),
        ("2026-05-10 14:30:00", "bob",     88.50),
        ("2026-05-10 20:05:00", "alice",  320.00),
        ("2026-05-12 11:00:00", "carol",   50.00),
        ("2026-05-12 18:45:00", "dave",  1200.00),
        ("2026-05-15 08:20:00", "alice",  299.99),
        ("2026-05-15 13:10:00", "eve",    150.00),
        ("2026-05-15 16:55:00", "bob",     75.25),
        ("2026-05-15 21:30:00", "carol",  800.00),
        ("2026-05-18 10:00:00", "dave",   450.00),
        ("2026-05-18 19:20:00", "eve",     99.99),
        ("2026-05-20 09:00:00", "alice", 1500.00),
```

```
        ("2026-05-20 12:30:00", "bob",    250.50),
        ("2026-05-20 17:45:00", "carol",   30.00),
    ];

    conn.exec_batch(
        "INSERT INTO orders (customer, amount, created_at)
         VALUES (:customer, :amount, :created_at)",
        orders.iter().map(|(ts, customer, amount)| {
            params! {
                "customer"   => customer,
                "amount"     => amount,
                "created_at" => ts,
            }
        }),
    )?;
    Ok(())
}

/// 按日期范围 [start, end] 查询每日报表。
fn query_report(pool: &Pool, start: NaiveDate, end: NaiveDate) -> Result<Vec<DailyReportRow>> {
    let mut conn = pool.get_conn()?;
    let rows = conn.exec_map(
```

```
    r"SELECT DATE_FORMAT(day, '%Y-%m-%d') AS day_str,

            order_count,

            CAST(total_amount AS CHAR) AS total_str,

            CAST(avg_amount   AS CHAR) AS avg_str

     FROM v_daily_sales_report

     WHERE day BETWEEN :start AND :end

     ORDER BY day",

    params! {

        "start" => start.format("%Y-%m-%d").to_string(),

        "end"   => end.format("%Y-%m-%d").to_string(),

    },

    |(day_str, order_count, total_str, avg_str): (String, i64, String, String)| {

        let day = NaiveDate::parse_from_str(&day_str, "%Y-%m-%d").unwrap();

        let total_amount = total_str.parse::<f64>().unwrap();

        let avg_amount = avg_str.parse::<f64>().unwrap();

        DailyReportRow { day, order_count, total_amount, avg_amount }

    },

  )?;

  Ok(rows)

}



fn main() -> Result<()> {
```

```
    let pool = Pool::new(DB_URL)?;

    init_schema(&pool)?;

    seed_orders(&pool)?;



    let start = NaiveDate::from_ymd_opt(2026, 5, 1).unwrap();

    let end = NaiveDate::from_ymd_opt(2026, 5, 31).unwrap();

    let rows = query_report(&pool, start, end)?;



    println!("销售报表 {start} ~ {end}（共 {} 天）：", rows.len());

    for r in rows {

        println!(

            "  {} | 订单数={} 销售额={:.2} 客单价={:.2}",

            r.day, r.order_count, r.total_amount, r.avg_amount

        );

    }

    Ok(())

}
```

分析：复用任务二的 mysql Pool，配合 chrono::NaiveDate 作为日期参数类型。init_schema 同时建立 orders 表和视图 v_daily_sales_report，视图把 COUNT(*)、SUM(amount)、AVG(amount) 与 GROUP BY DATE(created_at) 这一整套统计逻辑封装在数据库一侧，应用层只需 SELECT ... FROM v_daily_sales_report WHERE day BETWEEN :start AND :end 即可拿到聚合结果；

若日后统计有变，只需 CREATE OR REPLACE VIEW，无需重新部署应用代码。seed_orders 在每次启动时先 TRUNCATE 再用 exec_batch 批量插入 14 条订单，覆盖 5 个日期，使报表输出可复现。query_report 读取时刻意用 CAST(total_amount AS CHAR) 和 CAST(avg_amount AS CHAR) 在 SQL 端把 DECIMAL 转成字符串，再在闭包中 parse::<f64>()，绕开 mysql crate 默认把 DECIMAL 当作字节串返回、且 AVG 输出仍是 DECIMAL 类型导致 f64 直接绑定失败的问题。

四. 数据、结果分析

任务一：用户注册系统

```
=== 用户注册系统 (SQLite) ===

[1] 注册新用户
[2] 按用户名查询
[q] 退出
> 1
用户名: wuji
密码:   582192581
✓ 注册成功: id=6, username=wuji

[1] 注册新用户
[2] 按用户名查询
[q] 退出
> 2
要查询的用户名: alice
✓ 找到用户:
  id         = 1
  username   = alice
  password   = pass_alice
  created_at = 2026-05-20 15:48:17

[1] 注册新用户
[2] 按用户名查询
[q] 退出
> 2
要查询的用户名: wuji
✓ 找到用户:
  id         = 6
  username   = wuji
  password   = 582192581
  created_at = 2026-05-20 15:50:57

[1] 注册新用户
[2] 按用户名查询
[q] 退出
> q
再见。
```

结果分析：程序启动后默认存入五条用户信息。首先创建新用户，设置密码并进行 query，可以看到成功查询到刚刚创建的用户信息。查询提前存入的用户 alice，也可以看到数据库里的对应信息。

任务二：日志记录系统

```
最近 24 小时 ERROR 共 4 条：
  LogRow { log_id: 14, log_level: "ERROR", content: "刚刚：消息队列消费者掉线", created_at: 2026-05-20T20:16:17+08:00 }
  LogRow { log_id: 12, log_level: "ERROR", content: "3 小时前：支付回调签名校验失败", created_at: 2026-05-20T17:16:17+08:00 }
  LogRow { log_id: 10, log_level: "ERROR", content: "8 小时前：下游服务超时", created_at: 2026-05-20T12:16:17+08:00 }
  LogRow { log_id: 8, log_level: "ERROR", content: "18 小时前：数据库连接失败", created_at: 2026-05-20T02:16:17+08:00 }
```

结果分析：样本数据共 14 条，其中 ERROR 共 7 条，落在 24 小时窗口内的 ERROR 共 4 条。运行后程序输出最近 24 小时 ERROR 共 4 条，依次列出刚刚 3 小时前、8 小时前、18 小时前四条日志，按 created_at 倒序排列。窗口之外的两天前、26 小时前、25 小时前三条 ERROR 未出现在结果中，说明 NOW() - INTERVAL 24 HOUR 的边界过滤正确；其他级别的 INFO、WARN、DEBUG 日志同样被 log_level = 'ERROR' 过滤掉，与预期一致。

任务三：商品库存管理

```
=== 商品库存管理 (PostgreSQL) ===

[1] 查看所有商品
[2] 购买商品
[q] 退出
> 1
所有商品（共 5 件）：
  id=1   名称=面包     库存=13    更新于 2026-05-20 20:38:52
  id=2   名称=牛奶     库存=3     更新于 2026-05-20 20:38:52
  id=3   名称=鸡蛋     库存=30    更新于 2026-05-20 20:29:13
  id=4   名称=黄油     库存=5     更新于 2026-05-20 20:29:13
  id=5   名称=酸奶     库存=3     更新于 2026-05-20 20:29:13

[1] 查看所有商品
[2] 购买商品
[q] 退出
> 2
商品 id: 1
购买数量: 1
✓ 购买成功：商品 1 剩余库存 = 12
库存低于 10 件的商品（共 3 件）：
  id=5   名称=酸奶     库存=3     更新于 2026-05-20 20:29:13
  id=2   名称=牛奶     库存=3     更新于 2026-05-20 20:38:52
  id=4   名称=黄油     库存=5     更新于 2026-05-20 20:29:13

[1] 查看所有商品
[2] 购买商品
[q] 退出
> 2
商品 id: 1
购买数量: 3
✓ 购买成功：商品 1 剩余库存 = 9
库存低于 10 件的商品（共 4 件）：
  id=5   名称=酸奶     库存=3     更新于 2026-05-20 20:29:13
  id=2   名称=牛奶     库存=3     更新于 2026-05-20 20:38:52
  id=4   名称=黄油     库存=5     更新于 2026-05-20 20:29:13
  id=1   名称=面包     库存=9     更新于 2026-05-20 20:39:38
```

结果分析：启动后进入交互菜单，选 1 查看所有商品可看到面包、牛奶、鸡蛋、黄油、酸奶共 5 条记录，库存为多次试运行累积扣减后的当前值。选 2

购买商品 id=1 数量 1，提示购买成功并显示剩余库存 12，紧接着列出库存低于 10 件的商品共 3 件（酸奶、牛奶、黄油）。再次选 2 购买 id=1 数量 3，面包剩余库存降至 9，低库存清单新增面包一项，共 4 件。这表明 decrement_stock 的原子更新生效， list_low_stock 能正确按阈值筛选并按库存升序展示。

任务四：数据统计报表



结果分析：设定查询范围 2026-05-01 至 2026-05-31，运行后报表共 5 天：5-10 订单数 3、销售额 607.50、客单价 202.50；5-12 订单数 2、销售额 1250.00、客单价 625.00；5-15 订单数 4、销售额 1325.24、客单价 331.31；5-18 订单数 2、销售额 549.99、客单价 275.00；5-20 订单数 3、销售额 1780.50、客单价 593.50。各日数据与种子订单逐日加总核对完全一致，BETWEEN 范围过滤生效，视图中的 GROUP BY DATE(created_at) 把同一天的多笔订单聚合到同一行，符合实验要求。

五. 总结

本次实验围绕 Rust 生态下三种主流数据库的客户端 crate（rusqlite、mysql、postgres）展开，从最轻量的嵌入式 SQLite，到容器内的 MySQL 与

PostgreSQL，依次实现了用户注册、日志记录、库存管理和销售报表四个典型场景。

在写法上，四个任务都坚持参数化 SQL，无论是 ?N、:name 还是 $N 占位，所有外部输入都通过绑定接口送进驱动，避免拼接字符串引发的注入风险。建表语句统一放在 init_schema 中并使用 CREATE TABLE IF NOT EXISTS，让程序具备重复运行的幂等性；约束如 UNIQUE、ENUM、NOT NULL 尽量下沉到数据库层，减少应用代码的校验负担。任务三的库存扣减用一条带 WHERE stock >= $1 RETURNING stock 的 UPDATE 解决了判断与修改的并发问题，任务四则借助视图把统计逻辑沉淀在数据库端，使应用代码只承担取数和展示。

总体而言，通过本次实验，我对 Rust 调用关系型数据库的常规范式形成了较完整的认识，包括连接池、参数绑定、类型映射、错误传播以及跨数据库差异的处理方式，也加深了对约束放数据库、逻辑放视图、应用层只做粘合这一思路的理解。